Practice Questions

Author: Prof. Pranay Kumar Saha | Subject Code: **NCSC101**

1 Part 1 — Python Parser with Subcommands

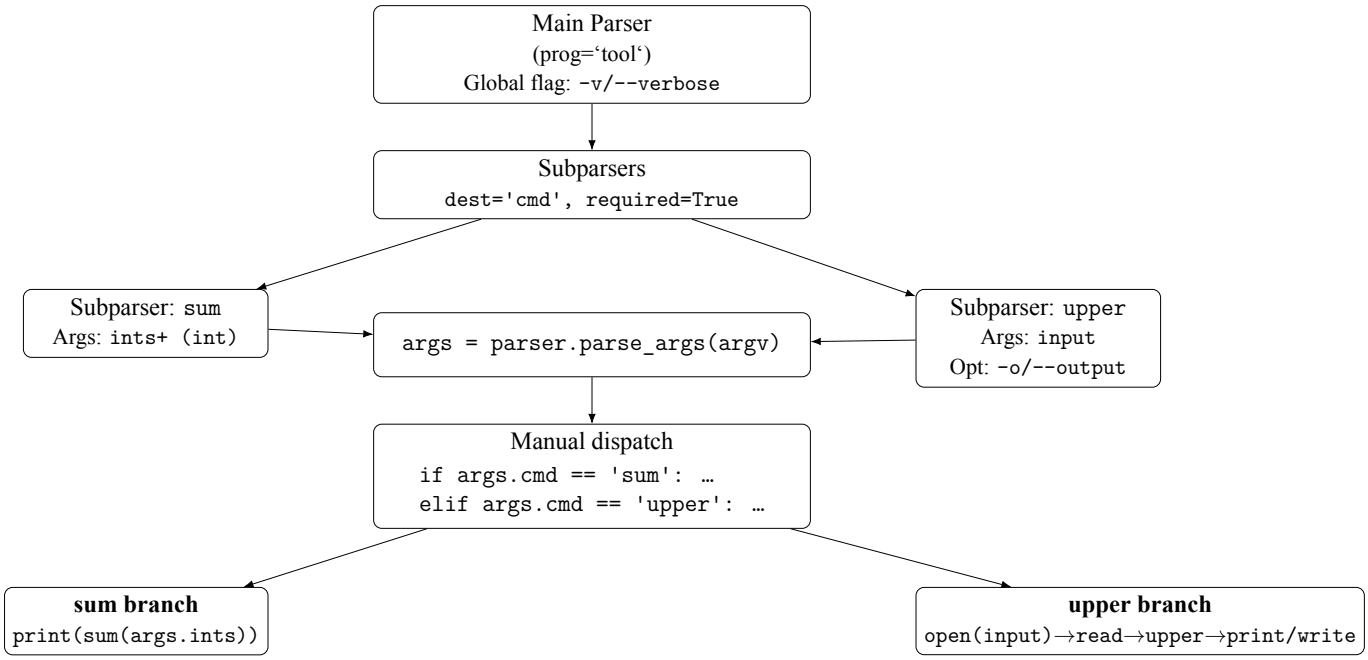
Problem: Design a CLI with two subcommands

Create `tool.py` with two subcommands:

- **sum** `ints...` → prints the sum of integers.
- **upper** `input [-o OUTPUT]` → uppercase the file to stdout or OUTPUT.

Also add a global flag `-v/--verbose`.

Parser block diagram



Solution & Explanation

Code (save as `tool.py`):

```
#!/usr/bin/env python3
"""
tool.py - minimal argparse subparsers demo
- One main() function
- No set_defaults()
```

```
- Manual dispatch using args.cmd (from dest="cmd")
"""

import argparse
import sys

def main(argv=None) -> int:
    # ---- build the top-level parser ----
    parser = argparse.ArgumentParser(
        prog="tool",
        description="Simple CLI with subcommands"
    )
    parser.add_argument(
        "-v", "--verbose",
        action="store_true",
        help="Enable verbose output"
    )

    # ---- add subparsers; store chosen name in args.cmd ----
    subs = parser.add_subparsers(dest="cmd")

    # subcommand: sum
    p_sum = subs.add_parser("sum", help="Sum integers")
    p_sum.add_argument(
        "ints", nargs="+", type=int, dest="ints",
        help="One or more integers"
    )

    # subcommand: upper
    p_upper = subs.add_parser("upper", help="Uppercase a file")
    p_upper.add_argument("input", dest="input", help="Input text
       file")
    p_upper.add_argument(
        "-o", "--output", dest="output",
        help="Write result to this file (prints to stdout if
           omitted)"
    )

    # ---- parse args ----
    args = parser.parse_args(argv)

    # If no subcommand was chosen, show help and exit with error
       code 2
    if args.cmd is None:
        parser.print_help(sys.stderr)
        return 2
```

```
    # ---- manual dispatch without set_defaults ----
    if args.cmd == "sum":
        # tool sum 1 2 3
        if args.verbose:
            print(f"[verbose] adding: {args.ints}", file=sys.
                stderr)
        print(sum(args.ints))
        return 0

    elif args.cmd == "upper":
        # tool upper input.txt [-o out.txt]
        if args.verbose:
            print(f"[verbose] reading: {args.input}", file=sys.
                stderr)

        # read input file
        try:
            with open(args.input, "r", encoding="utf-8") as f:
                text = f.read()
        except OSError as e:
            print(f"error: cannot read '{args.input}': {e}", file=
                sys.stderr)
            return 1

        # transform
        out_text = text.upper()

        # write or print
        if args.output:
            if args.verbose:
                print(f"[verbose] writing: {args.output}", file=
                    sys.stderr)
            try:
                with open(args.output, "w", encoding="utf-8") as f
                    :
                    f.write(out_text)
            except OSError as e:
                print(f"error: cannot write '{args.output}': {e}",
                    file=sys.stderr)
                return 1
        else:
            print(out_text, end="")
        return 0

    # Unknown subcommand (shouldn't happen with valid parsing)
```

```
    print(f"error: unknown command '{args.cmd}'", file=sys.stderr)
    return 2


if __name__ == "__main__":
    raise SystemExit(main())
```

Try it quickly:

```
python tool.py -h
python tool.py sum 2 3 4            # -> 9
printf "Hello\nWorld\n" > msg.txt
python tool.py upper msg.txt        # -> HELLO \n WORLD
python tool.py upper msg.txt -o out.txt && cat out.txt
```

Explanation:

1. `add_subparsers(dest="cmd", required=True)` forces choosing a subcommand.
2. Each subcommand gets its own arguments and handler via `set_defaults(func=...)`.
3. After `parse_args`, we call `args.func(args)` to dispatch.
4. File I/O uses plain `open()` for simplicity.

2 Part 2 — sed (10 min)

Problem: Normalize log lines and tag errors

You have `system.log`. Make all levels uppercase and prepend `FIXME:` to lines that contain `ERROR`. Edit in-place.

Solution & Explanation

Input (system.log before):

```
INFO: Boot
Warning: disk is slow
error: bad status
INFO: Ready
```

Commands:

```
# (a) Uppercase known levels
sed -i -E 's/\b(info|warning|error)\b/\U\1/g' system.log
```

What each part does

- `-i` **(in-place)**: edits `system.log` directly.[a]
- `-E`: enables *extended* regular expressions (so `( )` and `|` work without backslashes).
- `s/…/…/g`: substitution; replace all matches on each line (g = global).
- `\b`: word boundary (match whole words, not substrings like `catalog`).
- `(info warning |error)|`: alternation inside a capturing group; whatever matches becomes `\1`.
- `\U\1`: replace with the *uppercase* version of the captured text (GNU `sed`).

Output (system.log after):

```
INFO: Boot
WARNING: disk is slow
ERROR: bad status
INFO: Ready
```

Why it works: `\b` anchors whole words; `\U` uppercases the match; the second command rewrites only lines matching `ERROR`.

[a] On macOS/BSD `sed`, use `-i ''` (empty backup suffix): `sed -i '' -E '...'`

---

3 Part 3 — awk

Problem: Average score per user

Given `scores.txt`:

```
alice 80
bob 90
alice 88
carol 75
bob 92
alice 95
```

Compute and print `name average` for each user.

Solution & Explanation

Command:

```
awk '{sum[$1]+=$2; cnt[$1]++} END {for (k in sum) printf "%s %.4f\
   n", k, sum[k]/cnt[k]}' scores.txt
```

Example output (order may vary):

```
carol 75.0000
alice 87.6667
bob 91.0000
```

What each part does

- `awk`: runs the AWK text-processing program.
- `'{ ... } END { ... }'`: two program blocks:
  - *Main block* `{sum[$1]+=$2; cnt[$1]++}` runs **once per input line**.
  - *END block* `END { for (k in sum) ... }` runs **after all lines are processed**.
- `$1` and `$2`: first and second **fields** on each line (by default AWK splits on *whitespace*).
- `sum[$1]+=$2;`: add the second field (score) into an **associative array** bucket keyed by the first field (name). Equivalent to `sum[name] = sum[name] + score`.
- `cnt[$1]++;`: increment a **count** for that same key (`cnt[name]`).
- `END { for (k in sum) ... }`: after reading all lines, `for (k in sum)` iterates over all keys (names) seen.
- `printf "%s %.4f\n", k, sum[k]/cnt[k]`: print the key (name) and the **average** score with **4 decimal places**. Format:
  - `%s` → string (the name `k`)
  - `%.4f` → floating-point with 4 digits after the decimal
  - `\n` → newline
- `scores.txt`: the input file to read.

---

4 Part 4 — grep

Problem: Whole-word match, case-insensitive, with line numbers

Create `animals.txt`:

```
dog
Cat
wildcat
catfish
```

```
my cat is cute
catalog
CAT
```

Print only lines that contain the whole word `cat`, case-insensitive, with line numbers.

Solution & Explanation

Command:

```
grep -niw 'cat' animals.txt
```

Expected output:

```
2: Cat
5: my cat is cute
7: CAT
```

What each part does

- `grep`: searches lines in files that match a pattern (prints matching lines).
- `-n`: print **line numbers** with each matching line.
- `-i`: **case-insensitive** match (so `cat`, `Cat`, `CAT` all match).
- `-w`: match only a **whole word**. A word is a sequence of "word-constituent" characters (letters, digits, underscore). This prevents matching substrings like `wildcat`, `catfish`, `catalog`.
- `'cat'`: the search **pattern**. Here it's a plain literal; no regex metacharacters are used.
- `animals.txt`: the **input file** to search.
